Lab 04 — De Dockerfile: je eigen images bouwen

Theorie — het recept, de context, de cache, de twee duurste valkuilen, en wat er verandert met een build-engine zonder daemon.

Doelstellingen

- Begrijpen wat de **build context** is en waarom die bepaalt wat je kunt kopiëren.
- De essentiële instructies kennen en weten wat elk ervan oplevert.
- `CMD` van `ENTRYPOINT` onderscheiden, *shell*-vorm van *exec*-vorm.
- `ARG` van `ENV` onderscheiden.
- Een Dockerfile zo ordenen dat de **buildcache** zijn werk kan doen.

---

1. De build context

```
podman build -t mijn-api:1.0 .
```

De `.` aan het einde is geen versiering: dat is de **build context**, de map die de build mag lezen. Daar volgen twee harde regels uit:

- **Je kunt alleen kopiëren wat in de context staat.** `COPY ../secrets/sleutel.pem .` mislukt altijd: het bestand valt buiten de grens.
- **De hele map hoort bij de context**, ook `.git/`, `node_modules/`, `target/`, logbestanden en lokale configuratie. Met `COPY . .` belandt dat allemaal in de image.

PODMAN
Bij Docker verpakt de client de context in een archief voor de daemon — vandaar de regel `transferring context: 900MB` die Angular-builds zo traag maakt. **Buildah**, de ingebouwde build-engine van Podman, leest de map rechtstreeks: geen archief, geen overdracht. De context blijft wel de grens van wat je kunt kopiëren, en `.dockerignore` blijft onmisbaar — niet voor de snelheid, maar voor wat **in de image** belandt. Buildah aanvaardt ook de neutrale namen `Containerfile` en `.containerignore`.

Het bestand **`.dockerignore`** in de root van de context houdt buiten wat er niet in thuishoort:

```
.git
node_modules
target
.env
```

VALKUIL
Zonder `.dockerignore` neemt een `COPY . .` je lokale geheimen en de volledige `.git`-geschiedenis mee **in de uiteindelijke image** — leesbaar voor iedereen die ze downloadt. Een klassiek datalek.

De Dockerfile zelf mag ergens anders staan: met `-f docker/api.Dockerfile` wijs je hem aan.

2. De essentiële instructies

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine   # basisimage — altijd de 1e instructie
LABEL org.opencontainers.image.source="https://git.mijnbedrijf.be/betalingen/api"
WORKDIR /app                                # maakt de map aan en gaat erin
COPY target/api.jar /app/api.jar            # kopieert van de context naar de image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75"     # variabele aanwezig bij uitvoering
EXPOSE 8080                                 # documentatie: publiceert GEEN poort
USER 1000:1000                              # niet als root draaien
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]
```

Instructie	Rol	Bestandslaag?
`FROM`	Vertrekpunt	ja (de lagen van de basisimage)
`RUN`	Voert een commando uit **tijdens de build**	ja
`COPY` / `ADD`	Kopieert vanuit de context	ja
`WORKDIR`, `ENV`, `USER`, `EXPOSE`, `LABEL`	Metadata	nee (`0B`)
`CMD`, `ENTRYPOINT`	Wat er draait bij `run`	nee
`ARG`	Variabele **alleen voor de build**	nee

Drie dingen verdienen extra aandacht. **Kies `COPY`, niet `ADD`**: `ADD` pakt archieven ongevraagd uit en downloadt URL's — twee impliciete gedragingen. **`EXPOSE` publiceert niets**: publiceren doe je met `-p` (lab 07). **`RUN` draait tijdens de build** — niet bij het starten van de container.

> **ONTHOUDEN**
> Schrijf de `FROM` voluit: `docker.io/library/eclipse-temurin:21-jre-alpine`. Een kale `FROM eclipse-temurin:…` hangt af van hoe de bouwmachine geconfigureerd is (lab 02); in een bedrijf wordt dat `registry.intern/basis/…`.

3. `CMD` tegenover `ENTRYPOINT`

Beide bepalen wat er draait wanneer de container start. Het verschil zit in wat ze doen met de argumenten van `podman run`:

- **`CMD`** is een **standaardwaarde die je kunt vervangen**. `podman run mijn-image ander-commando` gooit de `CMD` overboord.
- **`ENTRYPOINT`** is het **vaste** programma. De argumenten van `podman run` komen er **achteraan**.

```
ENTRYPOINT ["java","-jar","/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

`podman run api` start `java -jar /app/api.jar --spring.profiles.active=prod`; `podman run api --spring.profiles.active=dev` start hetzelfde programma met het profiel `dev`. Dit is het standaardpatroon: `ENTRYPOINT` legt het programma vast, `CMD` levert de standaardargumenten. Debuggen kan altijd nog met `podman run --entrypoint sh -it mijn-image`.

→ SPRING BOOT
Argumenten na de JAR (`--spring.profiles.active=dev`, `--server.port=9090`) leest Spring als properties die voorrang krijgen op `application.yml`. Daarom is het duo `ENTRYPOINT` + `CMD` zo handig: één image, per omgeving een ander argument. In lab 08 zie je dat omgevingsvariabelen nog beter werken.

4. *Shell*-vorm en *exec*-vorm

Elk commando kun je op twee manieren schrijven, en dat is geen kwestie van smaak:

```
CMD java -jar /app/api.jar                # SHELL-vorm  -> /bin/sh -c "java -jar ..."
CMD ["java","-jar","/app/api.jar"]        # EXEC-vorm   -> java wordt PID 1
```

In *exec*-vorm **is** de applicatie PID 1: ze ontvangt `SIGTERM` en stopt netjes. In *shell*-vorm kruipt er een `/bin/sh` tussen — het probleem uit lab 03. Een shell die PID 1 blijft, geeft `SIGTERM` niet door; de applicatie krijgt het nooit, `stop` wacht tien seconden en maakt dan alles af.

Het sleutelwoord is **blijft**. Wat er echt gebeurt, hangt af van de shell in de image:

Geval	PID 1	`podman stop`
Exec-vorm	je applicatie	netjes, code 143
Shell-vorm, eenvoudig commando, basis **Alpine** (busybox)	je applicatie (de shell stapt opzij)	netjes, code 143
Shell-vorm, eenvoudig commando, basis **Debian/Ubuntu** (dash)	`/bin/sh`	10 s en dan code 137
Shell-vorm met een pipe, een `&`, een `;`	`/bin/sh`	10 s en dan code 137
Opstartscript dat de app start **zonder** `exec`	`/bin/sh`	10 s en dan code 137

→ LINUX / SHELL
`/bin/sh` is niet één programma: Debian en Ubuntu gebruiken `dash`, Alpine de `ash` van busybox. Sommige shells vervangen zichzelf door het commando wanneer dat het *laatste* van het script is (een impliciete `exec`); andere maken een kindproces aan en wachten. Daarom stopt dezelfde Dockerfile netjes op Alpine en niet op Debian.

ONTHOUDEN
Schrijf altijd de *exec*-vorm, met dubbele JSON-aanhalingstekens. Moet je toch via een shell, schrijf die dan expliciet **en** gebruik `exec`: `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]`.

Een gevolg dat vaak over het hoofd wordt gezien: in *exec*-vorm worden `$JAVA_OPTS`, `&&`, `|` en `>` **niet** geïnterpreteerd — er is geen shell om dat te doen.

5. `ARG` tegenover `ENV`

```
ARG VERSION=1.0            # alleen beschikbaar tijdens de build
ENV APP_VERSION=${VERSION} # blijft in de image en in de containers
```

	`ARG`	`ENV`
Zichtbaar tijdens de build	ja	ja
Aanwezig in de uiteindelijke image	**nee**	ja
Aanpasbaar bij de build	`--build-arg VERSION=2.0`	nee
Aanpasbaar bij uitvoering	nee	`podman run -e APP_VERSION=…`

VALKUIL
"`ARG` verdwijnt uit de image" betekent **niet** "`ARG` is veilig voor een geheim": de waarde blijft zichtbaar in `podman history` en in de buildcache. Een wachtwoord via `--build-arg` is een lek (lab 08).

6. De buildcache

De engine verwerkt de instructies in volgorde en cachet elk resultaat. Heeft hij precies deze instructie al eens uitgevoerd, bovenop dezelfde vorige laag, dan hergebruikt hij het resultaat (`--> Using cache`). Anders voert hij ze uit — **en vervalt de cache van alles wat erna komt**. Een instructie vervalt wanneer haar tekst verandert, wanneer de **inhoud** van gekopieerde bestanden verandert (`COPY` / `ADD`), of wanneer een eerdere instructie verviel. Vandaar de gouden regel: **van stabiel naar vluchtig**.

```
# SLECHT: de code verandert bij elke commit, dus alles wordt herbouwd
COPY . /app
RUN mvn dependency:go-offline

# GOED: de afhankelijkheden worden alleen opnieuw gedownload als pom.xml verandert
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

Voor Angular geldt dezelfde redenering: eerst `COPY package*.json` en `npm ci`, pas daarna `COPY .`. De winst loopt op tot minuten per CI-build — en ook de uitrol wordt sneller, want een ongewijzigde laag wordt bij `push` niet opnieuw verstuurd (lab 02). Alles herbouwen doe je met `--no-cache`.

7. Een correcte `RUN` schrijven

```
# SLECHT: drie lagen, een apt-cache van 40 MB meegeleverd in de image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOED: één enkele laag, effectieve opruiming
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
```

Opruimen moet **in dezelfde** `RUN` gebeuren als installeren: wat je in een latere laag verwijdert, wordt alleen verborgen, niet weggehaald (lab 02). En zet `apt-get update` nooit alleen in zijn eigen laag: wekenlang gecachet zou het verouderde pakketindexen blijven serveren — het probleem van *cache busting*.

8. In het bedrijf

De Dockerfile van een Spring Boot-backend ziet er zo uit (de eenvoudige versie; de multi-stage-versie volgt in lab 05):

```
FROM registry.intern/basis/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Let op de keuzes: een **JRE**-image in plaats van een JDK, uit de interne registry; een niet-root `USER`; *exec*-vorm; en een JAR uit `target/` — de Maven-build gebeurde dus al **eerder**, in de CI. Precies die beperking werkt multi-stage weg. Aan de Angular-kant containeriseer je nooit `ng serve`, wel het resultaat van `ng build`, geserveerd door nginx. Docker bouwt hetzelfde recept in de CI, Podman op jouw eigen machine: een Dockerfile is een standaard.

Onthouden

- De `.` van `build .` wijst de **context** aan: die bepaalt wat je kunt kopiëren en wat een `COPY . .` meesleept. `.dockerignore` is verplicht — ook met Podman, waar niets wordt overgedragen.
- `EXPOSE` documenteert, `-p` publiceert. `RUN` draait tijdens de build, `CMD` / `ENTRYPOINT` bij de uitvoering; `ENTRYPOINT` legt het programma vast, `CMD` levert vervangbare argumenten.
- *Exec*-vorm `["prog","arg"]`: de applicatie is PID 1 en ontvangt `SIGTERM`. *Shell*-vorm: het gedrag hangt af van de shell van de basisimage — dus vermijden. `ARG` leeft alleen tijdens de build, `ENV` blijft in de image — geen van beide is geschikt voor een geheim.
- Orden van stabiel naar vluchtig; één vervallen instructie doet alle volgende vervallen.
- Installeren en opruimen in **dezelfde** `RUN`, anders blijven de bestanden in de image zitten.

Woordenschat

build context: de map die de build mag lezen. — **`.dockerignore`** / **`.containerignore`**: wat uit de context wordt geweerd. — **Containerfile**: neutrale naam van de Dockerfile bij Podman. — **Buildah**: de build-engine van Podman. — **exec-/shell-vorm**: de twee schrijfwijzen van `CMD` / `ENTRYPOINT`. — **cache busting**: de cache bewust laten vervallen. — **basisimage**: de image die `FROM` aanwijst.